

CLAUDE.md Hierarchy & Claude Code Configuration

1. What is CLAUDE.md?

CLAUDE.md is a special markdown file that Claude Code reads automatically when it starts in a project. It gives Claude persistent instructions, context, and constraints without repeating them in every prompt. Think of it as Claude Code's standing orders for a project — everything in CLAUDE.md is injected into Claude's context at the start of every session.

2. The three-level hierarchy

Global — ~/.claude/CLAUDE.md

- Applies to ALL projects on your machine.
- Set here: personal preferences, coding style, output format defaults, tools you always want.
- Only you see this — it lives in your home directory.
- NEVER commit this to version control — it is personal.
- Lowest precedence — overridden by project and local.

Project — /project/CLAUDE.md

- Applies to the current project only.
- Set here: architecture decisions, coding standards, tech stack context, team conventions.
- COMMIT this to version control — the whole team benefits.
- Overrides global settings for this project.
- Middle precedence — overridden by local.

Local — /project/subdir/CLAUDE.md

- Applies only when Claude is working inside that specific subdirectory.
- Set here: subdirectory-specific conventions, specialist context, module-level overrides.
- Useful for monorepos with distinct sub-services (e.g. payments, auth, admin).
- HIGHEST precedence — overrides both project and global.
- Can be committed or kept local depending on team preference.

3. Precedence rules

When the same instruction appears at multiple levels, the most specific level wins. Think of it exactly like CSS specificity — more targeted = higher priority. All three levels are loaded and merged — they are NOT mutually exclusive.

Priority	Level	Path	Commit?
1 — Highest	Local	/project/subdir/CLAUDE.md	Optional
2 — Middle	Project	/project/CLAUDE.md	Yes — team shared
3 — Lowest	Global	~/.claude/CLAUDE.md	Never — personal

Exam trap — precedence question

- Question: 'Which CLAUDE.md takes precedence when the same rule appears at two levels?'
- Answer: Always the MORE specific level. Local > Project > Global.
- NOT the one loaded first — loading order is global → project → local.
- Loading order and precedence order are OPPOSITE. Load first = lowest priority.

4. What belongs in CLAUDE.md

Content type	Global	Project	Local
Personal coding style preferences	✓	—	—
Output format defaults	✓	—	—
Project architecture decisions	—	✓	—
Team coding standards	—	✓	—

Tech stack context	—	✓	—
Approved API / library lists	—	✓	—
Subdirectory-specific overrides	—	—	✓
Module-level compliance rules	—	—	✓
API keys, secrets, passwords	x Never	x Never	x Never

5. Custom slash commands

Claude Code lets you define custom `/commands` that trigger predefined workflows. They live in `.claude/commands/` and are written as markdown files. The filename becomes the command name.

Project commands — shared with team

- `.claude/commands/*.md` — committed to version control.
- Triggered by `/command-name` in any Claude Code session in the project.
- File format is markdown — natural language instructions, not code.
- Can reference `CLAUDE.md` context — standards defined there apply.
- Can accept arguments: `/review src/auth.ts` passes the filename as context.

Personal commands — private to you

- `~/.claude/commands/*.md` — in your home directory, never committed.
- Available in all projects on your machine.
- Useful for personal workflows that don't belong in team shared commands.

Never put in command files

- Executable code (Python, bash, etc.) — commands are markdown only.
- Secrets or API keys — same rule as `CLAUDE.md`.
- Hard-coded file paths that won't work on other machines.

Example command file — `.claude/commands/security-check.md`

```
# /security-check
```

```
Run a full security audit on the specified file:
```

1. Check for SQL injection vulnerabilities
2. Verify all inputs are sanitised
3. Confirm no secrets or API keys are hardcoded
4. Check authentication and authorisation logic
5. Output findings as a structured JSON report

6. CI/CD integration

Claude Code can run in headless mode inside CI/CD pipelines — no interactive terminal needed. This is the `--print` flag pattern.

The `--print` flag — the only CI flag you need to know

```
# Run Claude Code non-interactively in CI

claude --print "Review this PR for security issues and output findings as JSON"

# Common CI use cases

claude --print "Run all tests and report failures"

claude --print "Check if this migration is safe to apply"

claude --print "Verify the build output matches the expected schema"
```

Headless mode — `--print` flag

- `--print` makes Claude Code run non-interactively.
- Processes the prompt, outputs result to stdout, and exits.
- NOT `--ci`, NOT `--headless`, NOT `--auto`. Always `--print`.
- The exam will offer tempting wrong flag names — know it is always `--print`.

Exit codes

- Claude Code returns standard exit codes so CI systems can act on results.
- Exit 0 = success. Non-zero = failure.
- Pipelines can use these to gate deployments or fail the build.

CLAUDE.md in CI

- The project CLAUDE.md is read automatically in CI too.
- Your coding standards and constraints apply to automated runs just like interactive ones.
- No special configuration needed — it just works.

No human confirmation in CI

- In CI, Claude cannot pause and ask for human confirmation.
- Design prompts to be fully autonomous or pre-authorise specific actions explicitly.
- For irreversible actions in CI — bake the authorisation into the prompt itself.

7. Exam-critical summary table

Concept	Key rule
Global CLAUDE.md	~/.claude/CLAUDE.md — personal, all projects, never commit
Project CLAUDE.md	/project/CLAUDE.md — team-shared, commit to version control
Local CLAUDE.md	/project/subdir/CLAUDE.md — highest precedence, subdirectory override
Precedence order	Local > Project > Global (most specific wins)
Loading order	Global → Project → Local (opposite of precedence — a common trap)
Slash commands	.claude/commands/*.md — markdown files, natural language only
Personal commands	~/.claude/commands/*.md — private, not committed
CI/CD flag	--print flag — headless, stdout output, standard exit codes
Never in CLAUDE.md	Secrets, API keys, passwords, credentials — at any level

8. Quick recall — 3 things to remember

- 1 Three levels load and merge.**
They are not mutually exclusive. All three are read — conflicts resolve by specificity. Local wins over Project wins over Global.

2 Project CLAUDE.md gets committed. Global never does.
Project = team-shared, commit to version control. Global = personal preferences, never commit.

3 CI uses --print flag.
Headless mode, output to stdout, no interactive confirmations. The flag is --print — not --ci, --headless, or --auto.

9. Practice questions & answers

Three questions from today's session. Q3 was answered incorrectly — the --print flag is the most tested CI/CD fact in this domain. Lock it in.

Q1. A developer has: Global = '2-space indentation', Project = '4-space indentation', Local (/project/payments/) = 'tabs for indentation'. Working inside /project/payments/, which rule does Claude Code apply?

- A) 2-space — global takes precedence as the base configuration
- B) 4-space — project level overrides global
- ✓ **C) Tabs — local CLAUDE.md has the highest precedence**
- D) All three rules applied simultaneously causing a conflict error

Explanation:

- C is correct: When working inside /project/payments/, all three files load and merge. When the same rule conflicts, local wins — Local > Project > Global. Tabs specified in local overrides both project (4-space) and global (2-space).
- A is wrong: Global is the LOWEST priority, not the highest. It is the baseline everything overrides.
- B is wrong: Project does override global, but gets overridden itself by local.
- D is wrong: No conflict error. Levels are designed to override each other gracefully.

Q2. Your team wants a reusable `/security-check` command shared across the entire team. Where should the command file be placed and what format should it use?

- A) `~/claude/commands/security-check.md` — global directory, markdown format
- B) `.claude/commands/security-check.py` — project directory, Python script format
- ✓ C) **`.claude/commands/security-check.md` — project directory, markdown format**
- D) `/project/CLAUDE.md` — add the command definition directly inside `project CLAUDE.md`

Explanation:

- C is correct: Team-shared slash commands live in `.claude/commands/` at the project root and get committed to version control. File format is markdown — natural language instructions. The filename becomes the command name.
- A is wrong: `~/claude/commands/` is the global directory — private to one developer, not shared with the team.
- B is wrong: Command files are markdown, not Python scripts.
- D is wrong: `CLAUDE.md` is for persistent context and constraints, not for defining slash commands which need their own dedicated files.

Q3. A DevOps engineer wants to integrate Claude Code into GitHub Actions CI to automatically review every PR for security vulnerabilities. The review output must be captured by the pipeline and stored as a CI artifact. Which correctly describes how to run Claude Code?

Your answer: C

- A) Claude Code cannot run in CI pipelines — it requires an interactive terminal
- ✓ B) **Use `claude --print '...'` — headless mode outputs to stdout which the pipeline captures**
- C) Use `claude --ci '...'` — the dedicated CI flag handles artifact storage automatically
- D) Run Claude Code interactively and use a bot account to respond to confirmation prompts

Explanation:

- B is correct: The `--print` flag triggers headless mode. Claude Code processes the prompt, outputs to stdout, and exits with a standard exit code. The CI pipeline captures stdout like any other shell command.
- C is wrong (the trap): There is NO `--ci` flag in Claude Code. The exam invents plausible flag names to test whether you know the actual one. It is always `--print`.
- A is wrong: Claude Code absolutely runs in CI. `--print` is specifically designed for this.
- D is wrong: Using a bot for interactive prompts is an anti-pattern. `--print` eliminates the need for any interactive confirmation entirely.

Day 6 scorecard

2 / 3

Good session — one key flag to lock in

CLAUDE.md precedence rules	✓ Correct
Slash command placement & format	✓ Correct
CI/CD headless mode flag (--print)	✗ Review

Key takeaway from missed Q3 — CI/CD flag

- The CI/CD headless flag is --print. Not --ci, not --headless, not --auto.
- `claude --print '...'` → processes prompt → outputs to stdout → exits with standard exit code.
- The exam will offer --ci as a tempting distractor. It does not exist.
- Memory hook: --print because it PRINTS output to stdout for the pipeline to capture.

Next up: Day 7

Slash commands deep-dive and CI/CD workflows — advanced patterns, chaining commands, and pipeline integration.